

Hey, That’s My Data! Label-Only Dataset Inference in Large Language Models

Chen Xiong¹, Zihao Wang², Rui Zhu³, Tsung-Yi Ho¹, Pin-Yu Chen⁴,
Jingwei Xiong⁵, Haixu Tang², Lucila Ohno-Machado³

¹The Chinese University of Hong Kong, ²Indiana University Bloomington, ³Yale University, School of Medicine,
⁴IBM Research AI, ⁵University of California, Davis

Abstract—Large Language Models (LLMs) have revolutionized Natural Language Processing by excelling at interpreting, reasoning about, and generating human language. However, their reliance on large-scale, often proprietary datasets poses a critical challenge: unauthorized usage of such data can lead to copyright infringement and significant financial harm. Existing dataset-inference methods typically depend on log probabilities to detect suspicious training material, yet many leading LLMs have begun withholding or obfuscating these signals. This reality underscores the pressing need for label-only approaches capable of identifying dataset membership without relying on internal model logits.

We address this gap by introducing CatShift, a label-only dataset-inference framework that capitalizes on catastrophic forgetting—the tendency of an LLM to overwrite previously learned knowledge when exposed to new data. If a suspicious dataset was previously seen by the model, fine-tuning on a portion of it triggers a pronounced post-tuning shift in the model’s outputs; conversely, truly novel data elicits more modest changes. By comparing the model’s output shifts for a suspicious dataset against those for a known non-member validation set, we statistically determine whether the suspicious set is likely to have been part of the model’s original training corpus. Extensive experiments on both open-source and API-based LLMs validate CatShift’s effectiveness in logit-inaccessible settings, offering a robust and practical solution for safeguarding proprietary data.

1. Introduction

Large Language Models (LLMs) exhibit extraordinary capabilities in interpreting, reasoning about, and producing human language. Their influence has been transformative in numerous Natural Language Processing (NLP) applications, including text classification, sentiment analysis, and question answering. The proliferation of these models can be largely attributed to the abundant web-scale datasets utilized to enhance their performance. As LLMs continue to advance, however, issues surrounding data rights and licensing have grown increasingly urgent.

Nonetheless, this surge in development has also sparked debates regarding unauthorized usage of data. In particular, copyrighted content may be embedded in model training sets without proper consent, infringing on the rights of authors and potentially leading to financial harm [23], [34], [44]. A notable example is the New York Times filing a lawsuit against OpenAI and Microsoft over the alleged improper use of their copyrighted materials for training purposes [42]. Media companies, such as the New York Times, invest substantial resources in compiling these high-value datasets and typically reserve them for their own use rather than making them publicly accessible. Even in scenarios where certain datasets are released openly [14], [33], licensing terms generally confine their application to research and educational settings. Consequently, ensuring these data collections remain protected from unauthorized exploitation is of paramount importance. To meet this need, mechanisms for detecting unauthorized usage of proprietary datasets have become increasingly vital.

Challenges in Dataset Inference. Detecting the unauthorized usage of proprietary datasets in model training is paramount for safeguarding data ownership. A common strategy for addressing this issue is dataset inference [25], [27], in which dataset owners attempt to identify whether a given LLM has been trained on their data. Traditional approaches typically depend on logits or log probabilities returned by the target model. Their effectiveness arises from the overfitting tendency of LLMs, whereby a model exhibits higher confidence in data it has seen during training—reflected in its output distribution. However, these logit-based techniques are easily circumvented if the model provides only the final predicted tokens (without log probabilities). Indeed, Anthropic Claude [1] already withholds log probabilities in its outputs, making it difficult to establish whether a dataset is truly part of the training corpus and thus complicating efforts to protect copyright. As a result, new avenues are being explored to accomplish dataset inference even under stringent label-only conditions.

This challenge motivates our exploration of label-only dataset inference, which relies solely on the top-1 predicted token from the model. Compared to logit-based methods, label-only inference is more broadly applicable (including cases where log probabilities are unavailable) and shows

• The first three authors contributed equally to this work.

promise in being more robust against advanced defenses such as log probability perturbation [12]. A straightforward baseline might conclude that a dataset is a member if the model achieves strong performance on text completion; the underlying assumption is that seeing this dataset during training makes it more likely to be predicted correctly. However, as shown in Section 4.2, such a simple approach often fails in practice, prompting the development of more nuanced strategies.

Our Solution. To address these limitations, we propose leveraging the phenomenon of catastrophic forgetting (CF) in LLMs to better distinguish members from non-members. Catastrophic forgetting [24] occurs when a model learning new information overwrites previously learned knowledge. Our key insight is that if a suspicious dataset is indeed part of the model’s training set, then fine-tuning the model on a portion of it can help the model “remember” forgotten information, thereby mitigating CF. As a result, there will be a pronounced shift in the model’s outputs before and after fine-tuning. In contrast, if the suspicious dataset is not in the training set, the model’s output distribution remains relatively stable. Building on this idea, we utilize CF as an indicator of prior data exposure.

In this paper, we introduce CatShift, an approach that requires the target LLM to provide a fine-tuning API—a feature widely offered by major providers, including OpenAI (GPT-3.5), Hugging Face Transformers (for open-source models), Google Cloud Vertex AI (for PaLM-based solutions), and Claude 3 Haiku. Fine-tuning options are generally available because they enable users to incorporate domain-specific knowledge for improved performance and consistency. We also assume that the dataset owner has a validation set that the target LLM has not encountered, which is realistic if the owner regularly publishes new material that postdates the LLM’s training period. Given a suspicious dataset, CatShift begins by splitting it into training and testing subsets. We then fine-tune the target LLM using only the training subset of the suspicious dataset, expecting that if this dataset was previously part of the model’s training data, the model will “recall” partially forgotten information more readily. This recalling effect is characterized by a distinct and measurable change in the model’s outputs before and after fine-tuning on the suspicious dataset’s testing subset. Conversely, if the suspicious dataset never appeared in the model’s training corpus, the fine-tuning step introduces new information that was absent before, and the resulting output shifts tend to be more moderate. In the following, we detail how CatShift quantitatively distinguishes members from non-members through this CF-based mechanism.

To quantify this phenomenon, we focus on a hypothesis-testing-based methodology. Specifically, after fine-tuning, we compute scalar similarity scores (e.g., via a BERT-based scorer) between the model’s outputs before and after fine-tuning on the suspicious dataset. We then compare these scores to those obtained from a known non-member *validation set*, which we also fine-tune on and score in the same manner. Because the validation set is guaranteed not to have

appeared during the target model’s training, it provides a baseline distribution of score changes for non-member data.

By applying statistical tests such as the Kolmogorov–Smirnov or Mann–Whitney U to the similarity-score distributions of the suspicious dataset versus the validation set, we determine whether the suspicious dataset exhibits significantly different behavior than known non-members. If the computed p-value is below a specified threshold (e.g., 0.1), we conclude that the suspicious dataset’s distribution differs markedly from the validation baseline—indicating that the suspicious dataset was likely part of the model’s training corpus. This approach thus enables label-only membership inference by exploiting catastrophic forgetting and subsequent “relearning”, even when log probabilities are unavailable or have been obfuscated.

Results. We conduct a comprehensive evaluation of our approach across a diverse set of large language models, encompassing both publicly available checkpoints with well-documented training data (e.g., Pythia [2] and GPT-Neo [3]) as well as popular API-based models like GPT-3.5. For example, on the Pythia model, CatShift achieves an impressive AUC score of 0.979 and an F1 score of 0.863, significantly outperforming the baseline method, which obtains an AUC score of 0.707 and an F1 score of 0.717.

To further demonstrate the practical applicability of our method, we extend our evaluation to API-based models, such as GPT-3.5, where CatShift effectively distinguishes between member and non-member data. Specifically, when applied to the BookMIA dataset [37], CatShift achieves a p-value of 6.44×10^{-5} for member books, indicating strong evidence of membership, while the p-value for non-member books is 0.711, demonstrating the model’s ability to differentiate between in-domain and out-of-domain data.

While no prior label-only methods exist in the literature, our approach tackles this challenging setting by inferring membership status *without* requiring log probabilities or other privileged information about the target LLM’s internal states, thereby offering a robust route to identify potential copyright infringements in real-world applications.

In addition to strong empirical performance, our study offers new insights into why dataset inference is successful when framed under the concept of catastrophic forgetting. Specifically, we show that the degree of change in a model’s outputs before and after fine-tuning is closely tied to whether the data was part of the model’s original training set. Fine-tuning on data that the model has previously seen yields a more pronounced post-tuning shift, whereas genuinely novel data elicits more modest changes. This finding substantiates the underlying mechanism of our approach, reinforcing its applicability to identifying unauthorized usage of proprietary datasets in large language models.

Contributions. Our key contributions are outlined below:

- We conduct a comprehensive study of dataset inference in *label-only* settings, introducing the first method capable of handling real-world scenarios in which log probabilities and other privileged signals are unavailable. It is crucial for safeguarding proprietary data from unauthorized use.

- We present an inference approach built upon the phenomenon of catastrophic forgetting called CatShift. Through extensive experiments, we show that CatShift consistently outperforms baseline methods, underscoring its effectiveness in diverse settings.
- We provide both quantitative and qualitative evidence demonstrating that training members of a large language model undergo more significant output shifts post-fine-tuning compared to non-members. This finding sheds light on the underlying mechanisms driving successful dataset inference.

Roadmap. The rest of the paper is organized as follows: [Section 2](#) provides the background of our research; [Section 3](#) elaborates the CatShift technique; [Section 4](#) reports the evaluation results of CatShift; [Section 5](#) compares our work with prior related research; [Section 6](#) discusses the limitation of CatShift and the potential future directions, and [Section 7](#) concludes the paper.

2. Background

Before getting to the specifics of our dataset inference framework, we first provide background information on Membership Inference Attacks (MIA), Dataset Inference Attacks (DIA), Fine-Tuning APIs, and the underlying threat model of our research.

2.1. Membership Inference Attack

Membership inference attacks (MIAs) [35], [39], [46] are a form of privacy attack on machine learning models. In such an attack, an adversary with varying levels of access and knowledge about a target model f_{target} seeks to ascertain whether a specific data instance x was part of the model’s training set $\mathcal{D}_{\text{target}}^{\text{train}}$. Formally, the adversary constructs an attack model $f_{\text{attack}} : \mathcal{R} \rightarrow \{0, 1\}$, leveraging any available resources, such as model responses to queries, shadow models, partial or complete knowledge about the target model’s architecture, or subsets of the training data. Here, $\mathcal{R}$ represents the space of all possible responses from the target model, encompassing model outputs, parameters, or any related metadata.

The output of the attack model for a given instance x indicates the inferred membership status of x in the training set. An output of 1 signifies that x is inferred to be a member of $\mathcal{D}_{\text{target}}^{\text{train}}$ (i.e., it was used to train f_{target}), whereas an output of 0 implies that x is inferred to be a non-member (i.e., it was not used in training f_{target}).

2.2. Dataset Inference Attack

While membership inference attacks focus on determining whether a single data instance was used during training, *dataset inference attacks* instead seek to differentiate between data sampled from the target model’s training distribution, denoted by $p_{\mathcal{D}_{\text{train}}}$, and data drawn from a non-member distribution, $p_{\mathcal{D}_{\text{non-train}}}$. Building on the observation

by Maini et al. [27] that training data tends to reside farther from the decision boundaries in classification settings, these attacks utilize a distributional shift in features, logits, or other model outputs to infer membership on a broader scale.

Concretely, a scoring function $s(x)$ measures the distance of a given sample x from the model’s decision boundary or estimates its likelihood under the hypothesized training distribution. A threshold τ is then applied: if $s(x) > \tau$, x is categorized as belonging to $\mathcal{D}_{\text{train}}$; otherwise, x is inferred to be from $\mathcal{D}_{\text{non-train}}$. Recent work [25] extends this principle to large language models (LLMs), where fine-tuning on a dataset induces measurable shifts in token-level distributions. Hypothesis testing can thus be employed to perform membership inference, albeit at the cost of needing labeled member and non-member data to train a regression model, as well as a separate validation set to calibrate τ .

2.3. Fine-Tuning APIs

Fine-tuning APIs have become a widely provided feature in machine learning services, particularly for Large Language Models (LLMs). These APIs allow users to adapt pre-trained models to specific tasks or datasets, making them more useful in a wide range of applications. Fine-tuning offers a cost-effective way to personalize models, as users can leverage a powerful pre-trained base model and adjust it for their needs without the need for extensive retraining from scratch. This is particularly crucial in many scenarios where there is limited data or computational resources, or where users want to rapidly adapt a model to a new domain. For instance, OpenAI’s GPT-3.5 model can be fine-tuned on custom datasets for specialized use cases like customer service or medical information retrieval, without the need for starting from scratch.

Fine-tuning is fundamentally necessary in many settings because it allows for the integration of domain-specific knowledge into a model that was initially trained on a broad, general-purpose corpus. A concrete example of this is the application of fine-tuning in the healthcare domain, where general-purpose language models like BERT [10] or GPT models [4] can be fine-tuned on medical texts to improve their accuracy and relevance when performing tasks like medical text classification or extracting entities from clinical notes. Without fine-tuning, these models might fail to capture the nuances and terminology specific to the healthcare field, leading to inaccurate predictions. In a similar vein, legal firms use fine-tuned models to automatically analyze contracts, ensuring that the model understands the legal terminology and context.

Additionally, fine-tuning can help mitigate performance degradation when transferring models across different domains or populations, improving both accuracy and robustness. For example, a model trained on general data may not perform well when applied to texts from specific languages, regions, or cultures. In this context, fine-tuning allows for adaptation to the target domain, as shown by research on domain adaptation in NLP. For instance, models like BERT have been fine-tuned on regional dialects or specific datasets

such as medical or financial text corpora to improve their accuracy on tasks specific to those domains.

In settings like these, fine-tuning not only improves accuracy but also ensures that models remain robust across various contexts, preventing overfitting to one domain. A well-known example is the fine-tuning of LLMs for tasks like sentiment analysis, where pre-trained models are adapted to different sentiment lexicons or emotional nuances in diverse languages, improving their cross-lingual performance. Similarly, fine-tuning on specific user feedback or historical interactions has been shown to enhance chatbot performance in customer service applications, tailoring responses to reflect the company’s specific communication style and needs.

Fine-tuning offers the added advantage of enabling model creators to modify a model’s behavior without direct access to its internal parameters, making it more accessible to developers who might not have expertise in model training. This is especially important in industries where regulations around data privacy and intellectual property are stringent. For example, fine-tuning APIs like those provided by Hugging Face and OpenAI allow companies to comply with privacy regulations while still tailoring models to their specific use cases.

Thus, fine-tuning APIs provide a flexible and efficient mechanism for adapting and enhancing the capabilities of machine learning systems. By enabling users to modify a model’s behavior without requiring deep technical knowledge of its internals, these APIs democratize access to advanced AI technologies and expand their utility across a wide range of sectors.

2.4. Threat Model

Adversary and Goals. Our primary adversary is a Large Language Model (LLM) provider who may have incorporated proprietary data into their model’s training set without obtaining proper consent. The *dataset owner*, seeking to protect their copyrighted or otherwise sensitive data, aims to determine whether a suspicious dataset was used to train the target LLM. This adversary may attempt to conceal any unauthorized data usage, for instance by restricting log probability outputs or manipulating model responses. The ultimate goal of the dataset owner is thus to detect copyright infringement (or other illicit usage) under restrictive query conditions.

Defender Capabilities. We assume that the dataset owner:

- Has *black-box* access to the target LLM *only* via a label-only interface. That is, the owner can query the LLM with custom prompts but observes **only the top-1 output tokens** (or text), without receiving logits or log probabilities.
- Has the ability to *fine-tune* the target LLM on a custom dataset. Many commercial and open-source LLM providers (e.g., OpenAI GPT-3.5) offer fine-tuning APIs without disclosing the model’s internal parameters.
- Owns a known **validation set** that is guaranteed *not* to be in the LLM’s training corpus (e.g., newly created mate-

rial). This validation set provides a baseline for measuring output distribution shifts after fine-tuning on data believed to be novel to the model.

Adversarial Constraints. The LLM provider may impose several constraints to hide or obfuscate potential data usage:

- *Logit/Probability Obfuscation:* The provider may refuse to expose intermediate logits or log probabilities, thus preventing conventional logit-based inference techniques.
- *Output Manipulation:* The provider might attempt to alter or randomize token selection at inference time to mask overfitting signals or to reduce the consistency of the model’s predictions on suspicious data.
- *Fine-Tuning Limitations:* Fine-tuning options might be restricted in duration, the number of training steps, or model capacity changes. The provider could also charge fees that practically limit the number of fine-tuning experiments the dataset owner can conduct.

Our Assumptions. Despite these adversarial behaviors, we make the following assumptions to enable our CatShift approach:

- **Fine-Tuning Access:** The LLM provider’s API genuinely updates the model’s weights or parameters when fine-tuned on a dataset supplied by the owner. That is, fine-tuning is not a mere prompt-engineering trick but a procedure that modifies the model’s internal states.
- **Reliable Queries:** The owner can issue queries (both before and after fine-tuning) using the suspicious dataset’s testing subset, and the model will generate outputs consistent with its current state.
- **Validation Ground Truth:** The dataset owner can supply a validation dataset that is definitively known to be *absent* from the model’s training. This dataset provides a baseline for judging the typical amount of output shift induced by *truly novel* data during fine-tuning.

Attack Surface and Desired Protections. Under the above constraints, the dataset owner aims to detect unauthorized usage of their data, even if the model provider has taken steps to conceal overfitting signals. By using only label-only queries, the owner limits the provider’s ability to hide membership signals through log probability obfuscation. Additionally, by comparing fine-tuning behaviors on suspicious data versus known non-member data, the owner gains a statistical basis to distinguish between the two classes of datasets. Hence, the threat model centers on the owner’s ability to perform a robust dataset-inference test under minimal signals and potential adversarial interference from the LLM provider.

3. Methodology of CatShift

In this section, we describe the design and implementation of the CatShift framework, focusing on how we improve dataset inference using label-only outputs and assess the membership properties of a suspicious dataset.

3.1. Design Motivation

LLMs are increasingly deployed across diverse applications, often built using massive and potentially proprietary datasets. As these models expand their reach, it becomes critically important to determine whether a specific dataset has been used to train a model, particularly to uphold copyright and ensure legal compliance. Unfortunately, most existing methods for membership inference depend on outputs such as log probabilities or confidence scores, which many model providers now withhold to protect their intellectual property. This trend creates a pressing need for methods that can perform dataset inference using only *label-only* outputs, i.e., models that only return the top-1 predicted tokens or completions.

The core challenge in this label-only setting arises from the limited information that is available to us. Without access to more granular data, such as per-token likelihoods or the model’s internal activations, identifying whether a suspicious dataset was used in training the model becomes significantly harder. The key intuition behind our approach is based on the concept of *catastrophic forgetting*—a phenomenon where, during fine-tuning, a model can overwrite previously learned knowledge, but can also *reactivate* this knowledge if exposed again to data it has encountered during its initial training.

Building on this idea, we propose measuring the *shift* in the model’s output distribution when fine-tuning the model on a subset of the suspicious dataset. If the dataset was previously part of the model’s training data, the model will show a pronounced shift in its output distribution (e.g., top-1 predictions), as it “recovers” knowledge it had initially learned but forgotten during fine-tuning. On the other hand, if the suspicious data is entirely new, the shift in the model’s outputs will be much smaller, reflecting that the model is learning new patterns rather than reviving old ones. This shift allows us to distinguish between data that the model has seen before (member data) and data it has never encountered (non-member data).

Crucially, this method works entirely on label-only outputs, allowing it to function even when more detailed information like log probabilities is unavailable. By relying on shifts in top-1 predictions rather than complex internal signals, our approach offers a robust, simple, and broadly applicable method for dataset inference that is well-suited to the constraints of modern LLM services. Through this design, we hope to provide a solution that helps protect the intellectual property rights of dataset owners without requiring access to proprietary model internals.

3.2. Overview of CatShift

To address the label-only constraint effectively, CatShift follows a three-step process that integrates *catastrophic forgetting* detection with *statistical hypothesis testing*. Each step is designed to gather enough evidence to distinguish a *suspicious dataset* from a known *non-member validation*

set, all without requiring confidence scores or internal model states.

Step 1: Completion Prompt Construction. We begin by treating dataset inference as a text completion task. Specifically, each data sample is split into a *prompt* segment and a *completion* segment. We then embed the prompt segment within a generic instruction (e.g., “Complete the following text:”), ensuring that our approach requires only the model’s top-1 output tokens. This label-only requirement is compatible with most commercial LLM APIs, preserving realism for real-world applications.

Step 2: Target Model Fine-Tuning. Next, we fine-tune the target LLM on these prompt-completion pairs derived from the suspicious dataset. From the perspective of *catastrophic forgetting*, if the suspicious dataset was indeed in the model’s original training corpus, re-introducing it should “refresh” any forgotten internal representations, thus yielding a more pronounced shift in the model’s outputs. Conversely, truly novel data that the model has never seen before generally prompts a less drastic output shift, since the model must acquire entirely new patterns rather than revisiting old ones.

Step 3: Output Distribution Analysis. Finally, we measure how the model’s completions differ *before* and *after* fine-tuning. To accomplish this, we:

- 1) *Gather pre- and post-fine-tuning completions* for the suspicious dataset’s test subset.
- 2) *Compute similarity metrics* (e.g., lexical overlap or BERT-based scores) to quantify how much the outputs have changed.
- 3) *Contrast these changes against those of a known non-member validation set*, which is subjected to the same fine-tuning and evaluation pipeline to establish a distributional baseline for non-member data.

By applying a statistical hypothesis test to compare the suspicious set against the validation baseline, CatShift can conclude whether the suspicious dataset is likely part of the model’s original training data. Because we rely solely on top-1 predictions, this technique remains viable even when log probabilities or other rich signals are unavailable or deliberately obfuscated.

3.3. Completion Prompt Construction

We formulate each piece of text in the *suspicious dataset* as a prompt-completion pair. Specifically, let

$$D_s = \{(x_i, y_i)\}_{i=1}^N \quad (1)$$

denote the suspicious dataset, where x_i is the *prompt segment* and y_i is the *completion segment*. In our label-only paradigm, the target LLM returns only the continuation of x_i , which we denote by $\hat{y}_i$. We store $\hat{y}_i$ for subsequent analysis. This construction is compatible with standard LLM API usage, ensuring that the approach can be adopted by dataset owners attempting to validate potential unauthorized usage of their content.

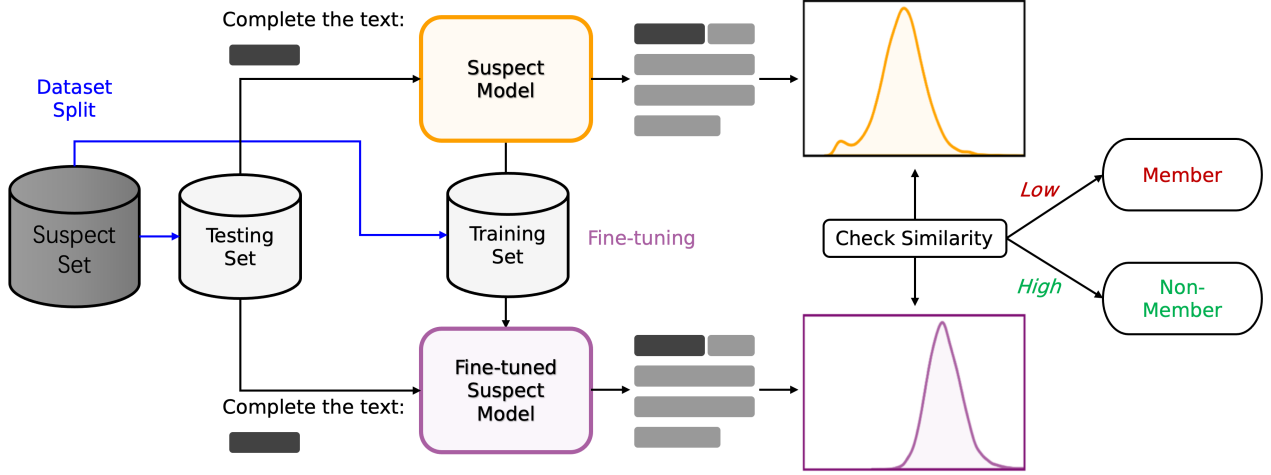


Figure 1: An overview of the CatShift framework for dataset inference on label-only large language models (LLMs). The figure illustrates the three-step process: (1) Dataset Split—where the suspicious dataset is partitioned into training and test subsets; (2) Completion Prompt Construction—where data samples are transformed into prompt-completion pairs, ensuring compatibility with label-only models; (3) Target Model Fine-Tuning—where the LLM is fine-tuned on the training subset of the suspicious dataset to detect shifts caused by re-exposing the model to previously seen data; (4) Output Distribution Analysis—where the model’s completions before and after fine-tuning are compared using similarity metrics, followed by statistical hypothesis testing to determine whether the suspicious dataset was part of the model’s original training data.

If the model has been fine-tuned through instruction-based training, we directly utilize the prompt to elicit the completion. Instruction-based training, or instruction tuning, involves adapting the model to respond effectively to explicit instructions, which often leads to more accurate and consistent completions when given a clear directive. In this case, the model is expected to generate completions that adhere to the instructions given in the prompt. However, if instruction tuning is not applied, we revert to using a prefix instead of a full prompt to trigger the continuation. This approach exploits the model’s dependency on the prefix for predicting the next token in a causal manner.

It is hypothesized that the model exhibits greater confidence on data it has encountered during training. When given a prefix, the causal model relies on it to predict the likelihood distribution of the next token. If the prefix originates from member data, the model is likely to replicate the data verbatim, even if that memorization is weak. This suggests that, for models not instruction-tuned, the use of a prefix can be an effective strategy for triggering model behavior that mimics the memorization of training data.

3.4. Target Model Fine-Tuning

We next fine-tune the target LLM, denoted $f^{(0)}$ with parameters $\theta^{(0)}$, on a portion of the suspicious dataset. Concretely, we partition D_s into:

$$D_s = D_s^{\text{train}} \cup D_s^{\text{test}}, \quad D_s^{\text{train}} \cap D_s^{\text{test}} = \emptyset. \quad (2)$$

Using D_s^{train} , we update the model parameters to obtain a fine-tuned variant $f^{(1)}$ with parameters $\theta^{(1)}$. Symbolically, this process can be represented as minimizing:

$$\theta^{(1)} = \arg \min_{\theta} \sum_{(x_i, y_i) \in D_s^{\text{train}}} \mathcal{L}(f^{(0)}(x_i; \theta), y_i), \quad (3)$$

where $\mathcal{L}$ is typically a cross-entropy loss.

- If D_s was originally part of the model’s training set, the model has already internalized relevant patterns of $x_i \rightarrow y_i$. Re-exposing the model to this data refreshes these patterns, potentially amplifying changes in subsequent predictions.
- If D_s is genuinely novel, the model must learn new patterns from scratch, generally leading to more gradual changes in its output distribution.

This distinction in how knowledge is “refreshed” versus “newly acquired” forms the basis for our later statistical test.

In our implementation, we leverage Low-Rank Adaptation (LoRA) [17] to efficiently fine-tune specific layers within the model architecture. LoRA allows for fine-tuning a low-rank decomposition of weight matrices, thus introducing minimal trainable parameters while maintaining computational efficiency. Specifically, we apply LoRA to fine-tune the following key components of the model:

- Query-Key-Value (QKV) Layer: The QKV layer plays a pivotal role in attention mechanisms. Fine-tuning this layer enables the model to adapt its attention weights to

more effectively capture patterns relevant to the suspicious dataset, whether these are remnants of previously seen data or new patterns requiring learning.

- **Linear Layer Following the QKV Layer:** This linear layer follows the attention mechanism and is crucial for translating the attention-based representations into final predictions. Fine-tuning this layer helps refine the mappings from internal representations to the final output space.
- **Multi-Layer Perceptron (MLP) Layer:** The MLP layer further processes the model’s activations before outputting predictions. Adjusting this layer allows for deeper modification of the model’s learned representations, facilitating both the “recovery” of previously learned knowledge and the integration of new data.

By focusing fine-tuning on these specific layers using LoRA, we efficiently update the model’s capabilities without incurring the overhead of adjusting all parameters. This targeted fine-tuning approach allows the model to rapidly adapt to the suspicious dataset while retaining the original knowledge stored in the pretrained model.

The key insight here is that fine-tuning these specific layers enables the model to reactivate relevant internal representations that were “forgotten” during the initial pretraining phase. If D_s is a member dataset, fine-tuning on D_s^{train} would likely lead to a significant shift in the model’s top-1 predictions, as the relevant representations are recovered. In contrast, if D_s is a non-member dataset, the model would learn novel patterns, leading to subtler shifts in its predictions. Thus, this selective fine-tuning strategy provides a way to isolate the impact of previously learned knowledge, facilitating the identification of dataset membership based on output shifts.

3.5. Output Distribution Analysis

We quantify the *shift* in the model’s outputs by comparing completions before and after fine-tuning. Let (x_i, y_i) be a test sample in D_s^{test} . We collect:

$$\hat{y}_i^{(0)} = f^{(0)}(x_i), \quad \hat{y}_i^{(1)} = f^{(1)}(x_i), \quad (4)$$

where $\hat{y}_i^{(0)}$ and $\hat{y}_i^{(1)}$ are top-1 completions from the original and fine-tuned models, respectively.

Similarity Score Computation. To assess how much the model’s outputs have changed, we define a similarity function $\text{Sim}(\cdot, \cdot)$, such as n -gram overlap or a BERT-based scoring function:

$$s_i = \text{Sim}(\hat{y}_i^{(0)}, \hat{y}_i^{(1)}). \quad (5)$$

A lower s_i indicates that the post-fine-tuning completion has diverged more substantially from the pre-fine-tuning version, suggesting a greater shift in the model’s output.

Comparison against Validation Baseline. To establish whether such output shifts are abnormally large, we compare them against those for a *known non-member validation set*, denoted

$$D_v = \{(x_j^v, y_j^v)\}_{j=1}^M, \quad (6)$$

which is guaranteed *not* to have appeared in the target model’s training data. We fine-tune and evaluate D_v following the same procedure:

$$\hat{y}_j^{(0)} = f^{(0)}(x_j^v), \quad \hat{y}_j^{(1)} = f^{(1)}(x_j^v), \quad (7)$$

and we compute

$$s_j^v = \text{Sim}(\hat{y}_j^{(0)}, \hat{y}_j^{(1)}). \quad (8)$$

This yields a baseline distribution for the magnitude of output shifts on data known to be *non-member*.

Hypothesis Testing. Let

$$S_s = \{s_i : i = 1, \dots, |D_s^{\text{test}}|\} \quad \text{and} \quad S_v = \{s_j^v : j = 1, \dots, M\} \quad (9)$$

denote the sets of similarity scores for the suspicious dataset and validation set, respectively. We apply a statistical test (e.g., Kolmogorov–Smirnov) to determine if these two distributions differ:

$$p\text{-value} = \text{KS}(S_s, S_v). \quad (10)$$

If $p\text{-value} < \alpha$ (for some threshold α , typically 0.1), we reject the null hypothesis that S_s and S_v come from the same distribution. Such a rejection suggests that the suspicious dataset D_s exhibits a distinctly larger output shift than the validation set and, hence, is more likely to be a training member. Conversely, failing to reject this null hypothesis indicates no strong statistical evidence to classify D_s as a training member.

By focusing on label-only completions and incorporating a robust validation baseline, CatShift enables accurate inferences about unauthorized dataset usage, even when model providers withhold log probabilities or apply other obfuscation strategies. This design underscores our method’s applicability and reliability in real-world settings, where limited access to a model’s internal confidence scores is the norm rather than the exception.

Overall Algorithm. The algorithm for the proposed CatShift framework can be found in [Algorithm 1](#).

3.6. Theoretical Analysis

In this section, we provide a concise theoretical perspective on why leveraging catastrophic forgetting (CF) can help distinguish training-member data from non-member data under a label-only setting. While the linkage between theory and the practical methodology may not be perfectly tight, the following analysis aims to capture the fundamental reasons behind the utility of CF in dataset inference tasks.

Setup. Consider a neural language model $f(\theta)$ with parameters $\theta \in \mathbb{R}^d$. We suppose that f initially undergoes *pre-training* on a dataset $\mathcal{D}_{\text{pre}}$. Let θ^* denote the parameter vector following this pre-training phase. Now, let $\mathcal{D}_{\text{sus}}$ denote a *suspicious* dataset, which we want to verify as either *member* (i.e., part of $\mathcal{D}_{\text{pre}}$) or *non-member*.

Catastrophic Forgetting in Fine-Tuning. After pre-training, the model is fine-tuned on some dataset $\mathcal{D}_{\text{ft}}$ (which

may or may not include $\mathcal{D}_{\text{sus}}$). Empirically, catastrophic forgetting suggests that certain parameters in θ^* adapted to $\mathcal{D}_{\text{sus}}$ could be *overwritten* if those parameters are needed for learning patterns from $\mathcal{D}_{\text{ft}}$. Denote by θ_{ft} the parameters after this fine-tuning. If $\mathcal{D}_{\text{sus}} \subset \mathcal{D}_{\text{pre}}$, we expect the model’s relevant “internal representation” for $\mathcal{D}_{\text{sus}}$ to have been diminished but *not* fully eliminated.

Subsequent Fine-Tuning on $\mathcal{D}_{\text{sus}}$. A key step in our methodology is fine-tuning the model again on $\mathcal{D}_{\text{sus}}$. Let $\theta_{\text{cf}}(\mathcal{D}_{\text{sus}})$ be the parameter vector after this reintroduction. Since $\mathcal{D}_{\text{sus}}$ was partially encoded in θ^* , the act of re-learning $\mathcal{D}_{\text{sus}}$ from θ_{ft} may *reactivate* latent parameter structures associated with $\mathcal{D}_{\text{sus}}$. Formally, consider a decomposition of the gradient update:

$$\nabla_{\theta} \mathcal{L}(f(\theta_{\text{ft}}), \mathcal{D}_{\text{sus}}) = \Delta_{\text{recover}} + \Delta_{\text{new}},$$

where Δ_{recover} captures gradient components that revive previously learned representations and Δ_{new} captures components for learning truly novel information.

If $\mathcal{D}_{\text{sus}}$ was present in the original training set $\mathcal{D}_{\text{pre}}$, Δ_{recover} should be larger, reflecting the reactivation of forgotten parameters. In contrast, if $\mathcal{D}_{\text{sus}}$ is entirely new, Δ_{recover} should be relatively small, with Δ_{new} dominating the overall update.

Impact on Label-Only Outputs. Even without access to log probabilities, the changes to the model parameters influence the top-1 predicted tokens. We can treat the distribution over possible completions as:

$$p_{\theta}(y | x) = \frac{\exp(\Phi_{\theta}(x, y))}{\sum_{y'} \exp(\Phi_{\theta}(x, y'))},$$

where $\Phi_{\theta}(x, y)$ is the model’s unnormalized logit score. Label-only outputs reveal only the $\arg \max$ of $p_{\theta}(\cdot | x)$. Yet, parameter updates that revive forgotten internal representations can cause abrupt changes in the top-1 completions. Conversely, parameter updates that mostly capture Δ_{new} result in subtler changes in how $\arg \max$ is computed, as the model must first learn new patterns rather than simply restoring existing ones.

Distributional Shift. By measuring some notion of *distance* (or *similarity*) between the model’s outputs before and after reintroduction of $\mathcal{D}_{\text{sus}}$, we capture how strongly Δ_{recover} influenced the final predictions. For a *non-member* dataset, there is no previously stored information to reactivate, causing a smaller shift in the output space. A statistical hypothesis test comparing suspicious-data shifts to those from a known non-member $\mathcal{D}_v$ can thus reveal whether the suspicious data “reactivates” old knowledge at a significantly higher level than expected.

Why This Works. At its core, the methodology exploits the fact that Δ_{recover} and Δ_{new} manifest differently in the final (label-only) outputs after fine-tuning:

- **Member data (Seen previously).** The “forgotten” representations can be recovered relatively quickly, causing large local changes in the model’s top-1 completions.

Algorithm 1 Dataset Inference with CatShift

- 1: **Input:** Suspicious dataset $D_s = \{(x_i, y_i)\}_{i=1}^N$, target model $f^{(0)}$ with parameters $\theta^{(0)}$
- 2: **Output:** Likelihood of dataset D_s being part of the training data
- 3: Split the suspicious dataset D_s into training and test subsets:

$$D_s^{\text{train}}, D_s^{\text{test}} = \text{Split}(D_s)$$

- 4: Construct completion function c .
- 5: Fine-tune the model $f^{(0)}$ on D_s^{train} to obtain model $f^{(1)}$:

$$\theta^{(1)} = \arg \min_{\theta} \sum_{(x_i, y_i) \in D_s^{\text{train}}} \mathcal{L}(f^{(0)}(x_i), y_i)$$

- 6: **for** each test sample $(x_i, y_i) \in D_s^{\text{test}}$ **do**
- 7: Compute top-1 completion from $f^{(0)}$: $\hat{y}_i^{(0)} = f^{(0)}(c(x_i))$
- 8: Compute top-1 completion from $f^{(1)}$: $\hat{y}_i^{(1)} = f^{(1)}(c(x_i))$
- 9: Compute similarity score $s_i = \text{Sim}(\hat{y}_i^{(0)}, \hat{y}_i^{(1)})$
- 10: **end for**
- 11: Compute scores for the known non-member validation set D_v :

$$s_j^v = \text{Sim}(\hat{y}_j^{(0)}, \hat{y}_j^{(1)}) \quad \text{for each } (x_j^v, y_j^v) \in D_v$$

- 12: Apply Kolmogorov-Smirnov test: $p\text{-value} = \text{KS}(S_s, S_v)$
 - 13: **Decision:**
 - 14: **if** $p\text{-value} < \alpha$ **then**
 - 15: Reject the null hypothesis and conclude that D_s is likely part of the model’s training data
 - 16: **else**
 - 17: Fail to reject the null hypothesis and conclude that D_s is not part of the model’s training data
 - 18: **end if**
-

- **Non-member data (Never seen).** The model lacks any pre-existing representation to “recover,” resulting in smaller, more gradual shifts in the top-1 completions.

While the above discussion does not perfectly map onto all practical details of implementing CatShift, it captures the essence of why reintroducing suspicious data can serve as a label-only test for membership. As long as catastrophic forgetting and recovery take place to a measurable extent, comparing the magnitude of the output shifts for suspicious data to those for a known non-member reference set provides a valid, theoretically grounded way to detect unauthorized usage of training data.

4. Evaluation

In this section, we present our experimental evaluation demonstrating the effectiveness of CatShift. We evaluate across multiple model scales and all the 22 subsets of the PILE dataset [14], as detailed in Section 4.1.

4.1. Experimental Setup

Datasets and Models. We conduct experiments on both public model checkpoints and API-based commercial mod-

els. The details of the datasets and models used are as follows:

For the public model checkpoints, we include the Pythia [2] and GPT-Neo [3] models. These models were trained on the PILE dataset [14], a large and diverse text corpus. This controlled experimental setup allows us to precisely analyze model behavior under conditions where membership status occurs independently and identically distributed (IID) without any temporal shifts, as both the training and validation splits of the PILE dataset are publicly accessible. This ensures that we can unambiguously identify the membership status of each sample: all samples in the training set are considered members, and all samples in the validation set are considered non-members. Notably, the validation splits of Pile are used as the non-member validation sets. Each target dataset has its own set.

For the API-based models, we use the fine-tuning interface for GPT-3.5 provided by OpenAI, which is designed to facilitate domain-specific tasks by adapting the model to specific datasets. However, since the membership status of these models is not publicly available, we use famous books listed in the BookMIA dataset [37] to evaluate membership. The BookMIA dataset contains both member and non-member samples from OpenAI models, making it suitable for our evaluation.

Unless otherwise noted, for each subset of the PILE dataset, our experiments compare 1000 samples from both the training and validation splits to evaluate membership. Additionally, for CatShift, we incorporate an additional 600 samples for fine-tuning.

Baseline. The baseline method does not involve fine-tuning the model. The core intuition behind the baseline approach is that if the model has been trained on a particular dataset, its ability to complete a given prompt will be highly accurate. In other words, when the model is provided with a segment from the dataset, the generated completion $\hat{y}_i$ will closely resemble the ground truth y_i for that dataset, as the model has internalized the patterns during training. Mathematically, for a given prompt x_i , the model produces a top-1 completion $\hat{y}_i = f(x_i)$. If the model has encountered the data during training, the similarity between the model’s output $\hat{y}_i$ and the true completion y_i should be high, i.e., $\text{Sim}(\hat{y}_i, y_i)$ will yield a large value.

Conversely, if the model has never seen the dataset before, its completion will likely be less accurate, and the discrepancy between $\hat{y}_i$ and y_i will be larger. The baseline method evaluates the accuracy of the model’s completions using a similarity score $\text{Sim}(\hat{y}_i, y_i)$, which is calculated using the BERT Score, similar with CatShift. Specifically, for a suspicious dataset $D_s = \{(x_i, y_i)\}_{i=1}^N$, the baseline measures the average similarity score for the top-1 predictions across all samples:

$$\bar{s}_b = \frac{1}{N} \sum_{i=1}^N \text{Sim}(\hat{y}_i, y_i)$$

If the model has encountered the data during training, we

expect $\bar{s}_b$ to be high, indicating that the model’s completions are close to the ground truth. If the dataset is novel, $\bar{s}_b$ will be lower, reflecting that the model’s completions are less accurate. In this sense, the baseline method provides a direct measure of the model’s familiarity with the dataset without the need for fine-tuning, relying on the assumption that a closer match to the ground truth suggests prior exposure to the data.

Evaluation Protocol. We apply CatShift to each subset of the PILE dataset, performing dataset inference separately for each subset. Specifically, we treat the provided *train* portion of each subset as candidate “member” data and the *validation* portion as candidate “non-member” data. For each experiment, we perform hypothesis testing using the Kolmogorov-Smirnov test, which compares the cumulative distribution functions (CDF) of the member and non-member data. The test statistic D is defined as:

$$D = \sup_x |F_n(x) - F_m(x)|$$

where $F_n(x)$ and $F_m(x)$ are the empirical CDFs of the non-member and member datasets, respectively, and $\sup_x$ represents the supremum over all values x . The p-value p is then computed based on the value of D , indicating the likelihood that the two datasets are drawn from the same distribution. A correctly attributed training dataset should yield p-values below the threshold of 0.1, while truly non-member subsets should produce p-values significantly above this threshold.

In addition to hypothesis testing, we also evaluate the performance of CatShift using the Receiver Operating Characteristic (ROC) Area Under the Curve (AUC) score and the F1 score, which provide a comprehensive view of classification performance. Each subset is labeled (0 for member, 1 for non-member), and its p-value is treated as a “score” for non-membership. The F1 score is computed by binarizing p-values with a threshold of 0.1.

These metrics are used to evaluate the overall performance of CatShift across the entire dataset.

CatShift Configuration. For fine-tuning, as previously mentioned, we use 600 data samples. For GPT-3.5, we apply the default fine-tuning configuration. For the Pythia and GPT-Neo models, we employ Low-Rank Adaptation (LoRA) [17] to fine-tune specific layers within the model architecture, including the query-key-value (QKV) layer, the linear layer following the QKV layer, and the multi-layer perceptron (MLP) layer. The LoRA configuration is parameterized with a rank of 8 and an alpha value of 32, and we apply a dropout rate of 0.1 to reduce overfitting. The fine-tuning process uses a learning rate of 8×10^{-5} and a batch size of 8. Checkpoints are saved every 10 steps, with the corresponding member and non-member fine-tuning losses recorded, summed, and normalized. The final model is selected based on the normalized losses calculated on the fine-tuning set. Note that the fine-tuning and testing sets do not overlap.

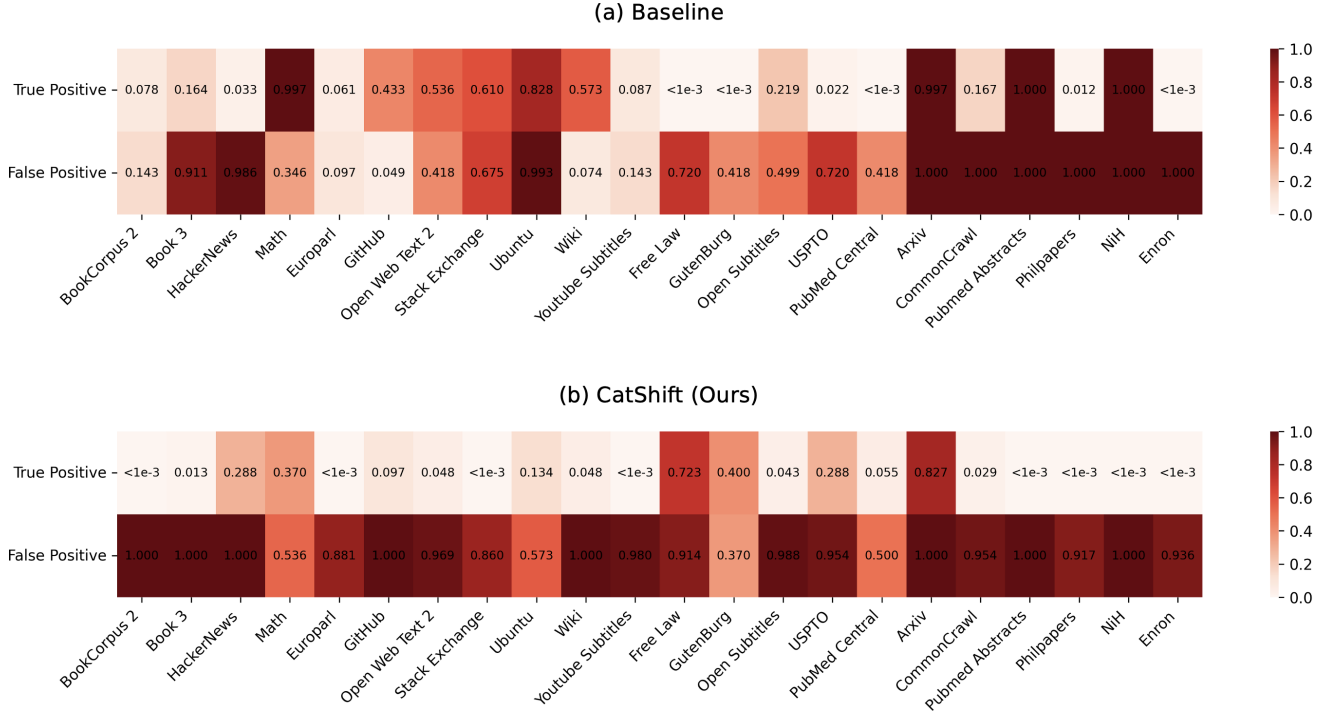


Figure 2: p-values of dataset inference by applying dataset inference to the Pythia-410M model with 1000 data points. We can observe that CatShift can correctly distinguish train and validation splits of the PILE with low p-values (usually below 0.1). Also, when considering false positives for comparing two validation subsets, we observe a p-value higher than 0.1 in all cases, indicating no false positives.

We build CatShift upon the baseline method, using a dual-test approach for evaluation. For each evaluation, we apply both CatShift and the baseline method to obtain distinct results. If the p-value from the baseline method is below the threshold of 1×10^{-3} , we accept its judgment and skip the application of CatShift. However, if the p-value from the baseline method exceeds the threshold of 1×10^{-3} , we disregard its judgment and apply CatShift for a second round of validation. In this case, the final decision is determined solely by CatShift.

4.2. Effectiveness of CatShift

In this section, we empirically validate the overall effectiveness of CatShift. Since most related works are designed for settings where logit access is available, we compare our solution exclusively with the baseline method, as no recent baselines specifically address logit-inaccessible settings like ours. In this section, the target model is fixed to Pythia-410M.

Figure 2 illustrates the p-values across different PILE subsets. Overall, we observe that CatShift significantly reduces the false positive rate compared to the baseline method in all settings. Specifically, when considering a p-value

threshold of 0.1, the baseline method results in false positives in 5 datasets. In contrast, CatShift results in *zero* false positives, highlighting its ability to more accurately distinguish between member and non-member datasets. Notably, A “true positive” is when the method correctly identifies a training subset (Pile’s train splits) as a member. A “false positive” is when the method incorrectly flags a non-member subset (Pile’s validation splits) as part of the training data.

Additionally, CatShift reduces the false negative rate more effectively than the baseline method across all settings. When applying a p-value threshold of 0.1, the baseline method shows false negatives in 7 datasets. In comparison, CatShift only shows false negatives in 3 datasets, suggesting a notable improvement in detecting true positives.

However, CatShift performs relatively worse on the “Math” subset. Despite this, even in the worst-case scenario, the p-values of non-members remain consistently higher than those of members. Specifically, the baseline method performs poorly in this subset, with non-members showing p-values around 0.346 and members showing p-values of 0.997. This indicates that the “Math” subset represents a generally challenging task. Nevertheless, CatShift still demonstrates a significant improvement over the baseline in this context.

When considering p-values under 1×10^{-3} , CatShift correctly identifies 6 datasets as true positives with p-values below this threshold. On the other hand, the baseline method only identifies 3 datasets as true positives under the same condition, emphasizing the superior sensitivity of CatShift to distinguishing members.

Similarly, when considering p-values greater than 0.9, CatShift correctly identifies 10 datasets as true negatives, with p-values above 0.9. In comparison, the baseline method only identifies 3 datasets as true negatives in this range, again demonstrating the higher specificity of CatShift.

In terms of overall performance, the baseline method achieves an AUC score of 0.707 and an F1 score of 0.717. In contrast, CatShift achieves a significantly higher AUC score of 0.979 and an F1 score of 0.863. This substantial improvement underscores the effectiveness of CatShift in accurately distinguishing between member and non-member datasets, particularly in settings where logit access is unavailable.

4.3. Impact of Target Model

In this experiment, we investigate the impact of the target model on the performance of CatShift. Specifically, we evaluate CatShift on three models: Pythia-410M, GPT-NEO-125M, and GPT-NEO-1.3B.

Figure 3 shows the p-values across different models. We observe that CatShift on the Pythia model achieves an extremely low false positive rate, which is zero when the p-value threshold is set to 0.1. In contrast, CatShift on the GPT-NEO models performs well in reducing the false negative rate, with all member p-values being below 1×10^{-3} for GPT-NEO-125M, and only three member datasets showing p-values above this threshold for GPT-NEO-1.3B.

However, when applied to the GPT-NEO models, the false positive rate increases. Specifically, we observe one false positive datasets for both GPT-NEO-125M and GPT-NEO-1.3B when the threshold is set to a p-value of 0.1.

In terms of performance, we find that the AUC score provides a more stable and reliable evaluation of dataset inference performance across different models. For calculating the F1-score, we use the 0.1 threshold for both Pythia and GPT-NEO models. Specifically, CatShift achieves an AUC score of 0.979 and an F1 score of 0.863 on the Pythia model. On the GPT-NEO-125M model, CatShift performs even better, with an AUC score of 0.994 and an F1 score of 0.957. For the GPT-NEO-1.3B model, CatShift achieves an AUC score of 0.941 and an F1 score of 0.898.

4.4. Effectiveness on Commercial Models

In this section, we investigate the effectiveness of CatShift on API-based commercial models, specifically focusing on OpenAI’s GPT-3.5. We utilize the fine-tuning interface provided by OpenAI, which is designed to adapt the model to domain-specific tasks by fine-tuning it on

custom datasets. For this evaluation, we use the default fine-tuning settings offered by OpenAI, ensuring that the model is trained according to the standard procedure.

However, since the membership status of these models is not publicly disclosed, we refrain from using the PILE dataset for evaluation, as we did in previous sections. Instead, we turn to the BookMIA dataset [37], which is specifically designed for evaluating membership status. The BookMIA dataset contains both member and non-member books from OpenAI models, making it an ideal candidate for assessing the membership prediction capabilities of CatShift.

The results show that CatShift achieves a p-value of 6.44×10^{-5} for member books, indicating strong evidence that these books belong to the model’s training data. In contrast, the non-member books have a p-value of 0.711, which is significantly higher, demonstrating that CatShift effectively distinguishes between member and non-member data. This clear separation in p-values highlights the robustness and applicability of CatShift when applied to commercial models, such as those offered by OpenAI, further validating its effectiveness in real-world scenarios.

5. Related Work

Membership Inference Attacks. Shokri et al. [39] pioneered the first membership inference method against black-box models by building multiple shadow models to mimic the target’s behavior and training separate attack models. Later, Salem et al. [35] eased key constraints assumed in [39], reducing the necessity of multiple shadow models, detailed knowledge of the target’s architecture, and a perfectly matching dataset. Yeom et al. [46] assumed that the adversary has information about both the distribution and size of the training set, along with collusion in the training procedure. Both [35] and [46] achieve results comparable to Shokri et al. [39]. Recent studies have also extended membership inference to various other domains, including natural language processing [5], [40], generative models [6], [15], [16], recommender systems [48], and federated learning [30], [32].

Several works [35], [39] have attributed the effectiveness of membership inference to the inherent “vanilla overfitting” in the target model. Consequently, some research leverages mechanisms that amplify this overfitting to further boost attack performance [7], [43]. However, no existing studies have capitalized on robustness overfitting for more advanced attacks. Moreover, it has been shown that adversarially robust models are unusually prone to membership inference [41], but these findings center on passive rather than active robustness overfitting. They also omit scenarios involving transfer learning, where any robust overfitting might not directly impact the original training data. Therefore, our work is the first to highlight the transferability of this threat, which remains insufficiently investigated.

While prior works have investigated label-only membership inference attacks (MIAs) [8], [21], [45], [47], they primarily focus on image-based models used in tasks such

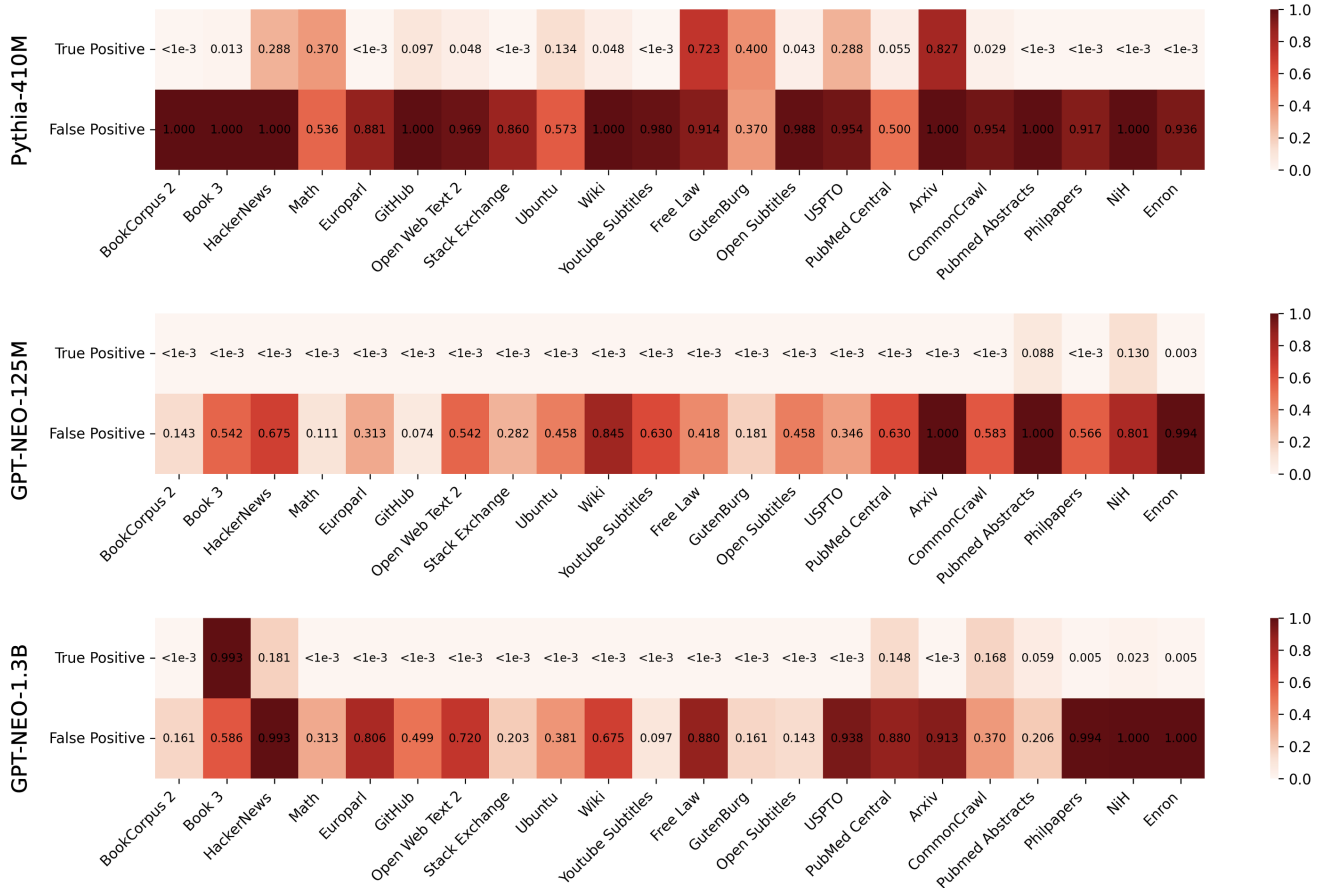


Figure 3: p-values of dataset inference by applying dataset inference to Pythia [2] and GPT-Neo [3] models with 1000 data points. We can observe that CatShift can correctly distinguish train and validation splits of the PILE with low p-values.

as classification or segmentation. These models differ fundamentally from large language models (LLMs) in both architecture and output modality, making the direct application of these approaches to LLMs impractical.

In contrast, membership inference in the context of LLMs presents distinct challenges—ranging from the non-deterministic nature of text generation to the lack of explicit labels—and is increasingly recognized as a separate line of inquiry [11]. As a result, direct comparisons between our work and prior label-only MIA techniques in the vision domain are not meaningful.

Our study instead focuses on dataset inference, which we believe has more immediate and practical implications—particularly in light of real-world concerns over unauthorized data use by LLMs [42]. Investigating membership inference tailored to LLMs is an important direction that we plan to pursue in future work.

MIA defenses. Most membership inference defenses aim to diminish the exploitable information leakage from the target model. Common approaches include obscuring confidence

scores [19], [22], [39], regularizing model parameters [20], [31], [39], or employing differential privacy [6], [18], [36].

Membership Inference Attacks in LLMs. MIA techniques, originally developed for machine learning classification models [39], have recently been adapted for LLMs. The baseline approach utilizes likelihood-based metrics, such as loss [46] and perplexity [5], to distinguish between members (documents used in LLM training) and non-members (documents not used in training). Several studies have demonstrated the effectiveness of likelihood-based MIAs in LLMs, achieving high AUC-ROC values. Other metrics, like Min-k%Prob [38], are based on the assumption that text previously encountered by the LLM is more likely to appear. Specifically, Min-k%Prob identifies the k% of tokens in a document with the lowest probabilities assigned by the LLM and computes their average log-likelihood.

Neighbor-based MIAs adjust the perplexity score using either neighboring models M' or neighboring documents

D' . Membership classification is then performed by comparing the likelihood of the target model M and document D with that of the neighboring model M' and document D , or the target model M and neighboring documents D' . The use of neighboring models requires access to a reference model trained on a disjoint dataset drawn from a similar distribution, which is often unrealistic [11]. While neighboring documents [13] offer a more realistic alternative, they generally yield lower performance and introduce challenges in properly setting noise parameters.

Recent studies [11], [26] question the high performance typically claimed for MIAs on LLMs. These studies identify several biases that can distort results, such as temporal shifts between member and non-member documents, leading to discrepancies in date distributions and word usage. Re-evaluating MIAs on Pythia [2], trained on genuinely random train/test splits from The Pile [14] (which eliminates bias), has shown a decrease in AUC-ROC scores, challenging the apparent success of some MIAs.

Some studies suggest that naive classifiers can effectively distinguish between members and non-members based on these biases [9], [28]. These findings conclude that MIAs should be evaluated using random datasets from the same distribution. While this is feasible with open LLMs like Pythia, which disclose their training and test data sources, it is not possible for LLMs that do not reveal their sources (such as those involved in copyright disputes). As noted in the literature [11], [26], [29], MIAs can exhibit varying performance (accuracy and relevance) across different LLMs and datasets. Consequently, MIAs' evaluation on open LLMs cannot be directly generalized to LLMs that do not disclose their training datasets.

6. Discussion

Limitations. Despite the strengths of CatShift, several practical constraints and open questions remain. First, catastrophic forgetting may be weaker or more difficult to detect for certain classes of datasets, especially those that overlap substantially with other data in the model's original training corpus. In such scenarios, it may be challenging to disentangle shifts caused by *true reactivation* of learned knowledge from those arising due to partial redundancy with content already present in the model's parameters. Second, the fine-tuning procedure itself introduces hyperparameters (e.g., learning rate, number of epochs, or choice of layers to update) that can significantly affect the observed output shifts. Determining optimal hyperparameter configurations for membership inference, especially under limited budget constraints, remains an open research question. Finally, while our label-only approach targets modern APIs where log probabilities are withheld, providers may deploy additional strategies (e.g., heavy output truncation or specialized filters) that can further limit the information available to dataset owners, thereby complicating or weakening the method's efficacy.

Future Work. Moving forward, one promising direction is to explore *partial membership* inference, where only a subset of the suspicious dataset appears in the original training corpus. Understanding how partial overlap influences catastrophic forgetting and reactivation could reveal more nuanced detection capabilities. Another avenue of research is investigating more advanced statistical testing frameworks or *ensemble methods*, which could consolidate evidence from multiple similarity metrics or complementary datasets to improve robustness. Additionally, extending CatShift to scenarios with *stronger adversarial defenses* (e.g., where the provider deliberately introduces perturbations to completions) would broaden its applicability in real-world settings. Finally, developing theoretical guarantees that more tightly connect the catastrophic forgetting process with detectable output shifts—potentially through novel optimization bounds or parameter-space analyses—may offer deeper insight into the fundamental limits of label-only membership inference.

Ethical Considerations. We followed established ethical guidelines and best practices throughout this study. Most evaluations were conducted in a controlled local environment using publicly available datasets. For the GPT-3.5 experiments, we used the fine-tuning API with data that were also publicly available for both fine-tuning and inference. The resulting fine-tuned model was not made publicly accessible, thereby minimizing potential ethical concerns. As the study relies solely on publicly available data and does not involve human participants, it was deemed not to constitute human subjects research by our Institutional Review Board (IRB). While we acknowledge the possibility of misuse of our findings, we emphasize the importance of highlighting these issues to foster awareness and encourage constructive dialogue.

Open Science. In keeping with open science principles, we will release all source code developed in this research upon acceptance of the paper. Should significant concerns regarding potential misuse arise, we will implement appropriate measures to limit access to the code.

7. Conclusion

In this paper, we have introduced CatShift, a label-only dataset-inference framework that leverages catastrophic forgetting to detect whether a suspicious dataset was used in training a large language model. By fine-tuning the model on a portion of this suspicious dataset and comparing output shifts before and after fine-tuning with a known non-member validation set, CatShift reliably identifies proprietary data usage even when log probabilities are unavailable. Our experiments on both open-source and commercial LLMs highlight the robustness and practicality of this approach, offering a vital safeguard against unauthorized data use while revealing promising directions for further research in label-only dataset inference.

References

- [1] Anthropic. The claude 3 model family: Opus, sonnet, haiku. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf, March 2024.
- [2] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 2397–2430. PMLR, 2023.
- [3] Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. GPT-Neo: Large-Scale Autoregressive Language Modeling with Mesh-TensorFlow, 2021. Software available from Zenodo.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [5] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom B. Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 2633–2650, 2021.
- [6] Dingfan Chen, Ning Yu, Yang Zhang, and Mario Fritz. Gan-leaks: A taxonomy of membership inference attacks against generative models. In *CCS ’20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 343–362, 2020.
- [7] Yufei Chen, Chao Shen, Yun Shen, Cong Wang, and Yang Zhang. Amplifying membership exposure via data poisoning. In *NeurIPS*, 2022.
- [8] Christopher A Choquette-Choo, Florian Tramèr, Nicholas Carlini, and Nicolas Papernot. Label-only membership inference attacks. In *International conference on machine learning*, pages 1964–1974. PMLR, 2021.
- [9] Debeshee Das, Jie Zhang, and Florian Tramèr. Blind baselines beat membership inference attacks for foundation models. *arXiv preprint arXiv:2406.16201*, 2024.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [11] Michael Duan, Anshuman Suri, Niloofar Mireshghallah, Sewon Min, Weijia Shi, Luke Zettlemoyer, Yulia Tsvetkov, Yejin Choi, David Evans, and Hannaneh Hajishirzi. Do membership inference attacks work on large language models? *arXiv preprint arXiv:2402.07841*, 2024.
- [12] Hao Fang, Yixiang Qiu, Hongyao Yu, Wenbo Yu, Jiawei Kong, Baoli Chong, Bin Chen, Xuan Wang, and Shu-Tao Xia. Privacy leakage on dnns: A survey of model inversion attacks and defenses. *CoRR*, abs/2402.04013, 2024.
- [13] Filippo Galli, Luca Melis, and Tommaso Cucinotta. Noisy neighbors: Efficient membership inference attacks against llms. *arXiv preprint arXiv:2406.16565*, 2024.
- [14] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *CoRR*, abs/2101.00027, 2021.
- [15] Jamie Hayes, Luca Melis, George Danezis, and Emiliano De Cristofaro. LOGAN: evaluating privacy leakage of generative models using generative adversarial networks. *CoRR*, abs/1705.07663, 2017.
- [16] Benjamin Hilprecht, Martin Härterich, and Daniel Bernau. Monte carlo and reconstruction membership inference attacks against generative models. *Proc. Priv. Enhancing Technol.*, 2019(4):232–249, 2019.
- [17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [18] Bargav Jayaraman and David Evans. Evaluating differentially private machine learning in practice. In *28th USENIX Security Symposium, USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019*, pages 1895–1912, 2019.
- [19] Jinyuan Jia, Ahmed Salem, Michael Backes, Yang Zhang, and Neil Zhenqiang Gong. Memguard: Defending against black-box membership inference attacks via adversarial examples. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, pages 259–274, 2019.
- [20] Yigitcan Kaya and Tudor Dumitras. When does data augmentation help with membership inference attacks? In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, pages 5345–5355, 2021.
- [21] Zheng Li and Yang Zhang. Membership leakage in label-only exposures. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 880–895, 2021.
- [22] Zheng Li and Yang Zhang. Membership leakage in label-only exposures. In *CCS ’21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, pages 880–895, 2021.
- [23] Sijia Liu, Yuanshun Yao, Jinghan Jia, Stephen Casper, Nathalie Baracaldo, Peter Hase, Xiaojun Xu, Yuguang Yao, Hang Li, Kush R. Varshney, Mohit Bansal, Sanmi Koyejo, and Yang Liu. Rethinking machine unlearning for large language models. *CoRR*, abs/2402.08787, 2024.
- [24] Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *CoRR*, abs/2308.08747, 2023.
- [25] Pratyush Maini, Hengrui Jia, Nicolas Papernot, and Adam Dziedziec. LLM dataset inference: Did you train on my dataset? *CoRR*, abs/2406.06443, 2024.
- [26] Pratyush Maini, Hengrui Jia, Nicolas Papernot, and Adam Dziedziec. Llm dataset inference: Did you train on my dataset? *arXiv preprint arXiv:2406.06443*, 2024.
- [27] Pratyush Maini, Mohammad Yaghini, and Nicolas Papernot. Dataset inference: Ownership resolution in machine learning. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.
- [28] Matthieu Meeus, Shubham Jain, Marek Rei, and Yves-Alexandre de Montjoye. Inherent challenges of post-hoc membership inference for large language models. *arXiv preprint arXiv:2406.17975*, 2024.
- [29] Matthieu Meeus, Igor Shilov, Manuel Faysse, and Yves-Alexandre de Montjoye. Copyright traps for large language models. In *41st International Conference on Machine Learning*, 2024.
- [30] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. Exploiting unintended feature leakage in collaborative learning. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*, pages 691–706, 2019.

- [31] Milad Nasr, Reza Shokri, and Amir Houmansadr. Machine learning with membership privacy using adversarial regularization. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, pages 634–646, 2018.
- [32] Milad Nasr, Reza Shokri, and Amir Houmansadr. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*, pages 739–753, 2019.
- [33] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67, 2020.
- [34] Jie Ren, Han Xu, Pengfei He, Yingqian Cui, Shenglai Zeng, Jiankun Zhang, Hongzhi Wen, Jiayuan Ding, Hui Liu, Yi Chang, and Jiliang Tang. Copyright protection in generative AI: A technical perspective. *CoRR*, abs/2402.02333, 2024.
- [35] Ahmed Salem, Yang Zhang, Mathias Humbert, Pascal Berrang, Mario Fritz, and Michael Backes. MI-leaks: Model and data independent membership inference attacks and defenses on machine learning models. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*, 2019.
- [36] Virat Shejwalkar and Amir Houmansadr. Membership privacy for machine learning models through knowledge transfer. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pages 9549–9557, 2021.
- [37] Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. Detecting pretraining data from large language models, 2023.
- [38] Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. Detecting pretraining data from large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [39] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 3–18, 2017.
- [40] Congzheng Song and Vitaly Shmatikov. Auditing data provenance in text-generation models. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, pages 196–206, 2019.
- [41] Liwei Song, Reza Shokri, and Prateek Mittal. Privacy risks of securing machine learning models against adversarial examples. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, pages 241–257, 2019.
- [42] The New York Times. The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work. <https://www.nytimes.com/2023/12/27/business/media/new-york-times-open-ai-microsoft-lawsuit.html>, December 2023.
- [43] Florian Tramèr, Reza Shokri, Ayrton San Joaquin, Hoang Le, Matthew Jagielski, Sanghyun Hong, and Nicholas Carlini. Truth serum: Poisoning machine learning models to reveal their secrets. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 2779–2792, 2022.
- [44] Johnny Tian-Zheng Wei, Ryan Yixiang Wang, and Robin Jia. Proving membership in LLM pretraining data via data watermarks. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, pages 13306–13320. Association for Computational Linguistics, 2024.
- [45] Yutong Wu, Han Qiu, Shangwei Guo, Jiwei Li, and Tianwei Zhang. You only query once: an efficient label-only membership inference attack. In *The Twelfth International Conference on Learning Representations*, 2024.
- [46] Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. Privacy risk in machine learning: Analyzing the connection to overfitting. In *31st IEEE Computer Security Foundations Symposium, CSF 2018, Oxford, United Kingdom, July 9-12, 2018*, pages 268–282, 2018.
- [47] Guangsheng Zhang, Bo Liu, Tianqing Zhu, Ming Ding, and Wanlei Zhou. Label-only membership inference attacks and defenses in semantic segmentation models. *IEEE Transactions on Dependable and Secure Computing*, 20(2):1435–1449, 2022.
- [48] Zhihao Zhu, Chenwang Wu, Rui Fan, Defu Lian, and Enhong Chen. Membership inference attacks against sequential recommender systems. In *Proceedings of the ACM Web Conference 2023, WWW 2023, Austin, TX, USA, 30 April 2023 - 4 May 2023*, pages 1208–1219, 2023.